

Sicherheitskonzept

Secure-by-Design-Zielarchitektur für einen LLM-basierten Helpdesk-Agenten mit MCP-Anbindung

Franziska Albrecht · Franziska.Sophie.Albrecht@proton.me

github.com/Franzer82/secure-agent-lab

Stand: 9. Oktober 2026

Bezieht sich auf Agent-Version 2.1. Alle Daten im Labor sind frei erfunden.

1. Zweck, Geltungsbereich und Status

Dieses Konzept beschreibt, wie ein KI-Agent mit Werkzeugzugriff über das Model Context Protocol (MCP) gegen die typischen Risiken von LLM-Anwendungen abgesichert wird: Prompt Injection, Abfluss vertraulicher Daten, überprivilegierte Agenten und ein vergiftetes Gedächtnis. Grundlage ist ein Laborprojekt. Ein IT-Helpdesk-Agent der fiktiven Nordlicht Software GmbH wurde zuerst absichtlich verwundbar gebaut (Version 1), systematisch angegriffen, gehärtet (Version 2) und nachgebessert (Version 2.1).

Das Konzept richtet sich an Architektur, Informationssicherheit, Datenschutz und Betrieb. Es nennt zu jeder Maßnahme, wogegen sie wirkt, auf welcher Schicht sie sitzt und was darüber belegt ist.

Status der Belege. Maßnahmen der Version 2 wurden mit dem Modell gemessen (siehe Assessment-Bericht). Was erst Version 2.1 hinzufügt, ist durch automatische Tests und die Wiedergabe gespeicherter Aufrufe geprüft, aber nicht im Betrieb mit dem Modell gemessen. Das Konzept kennzeichnet das bei jeder Maßnahme.

Im Geltungsbereich	Nicht im Geltungsbereich
Agent (Gesprächsschleife, Policy, Ausgabefilter), MCP-Server mit sechs Werkzeugen, Wissensdokumente, Gedächtnis, Protokollierung	Plattform und Modell des Anbieters, Betriebssystem, Netzwerk. Chat-Templates selbst betriebener Modelle sind nicht anwendbar, weil das Modell als verwaltete API genutzt wird.

2. Systembeschreibung

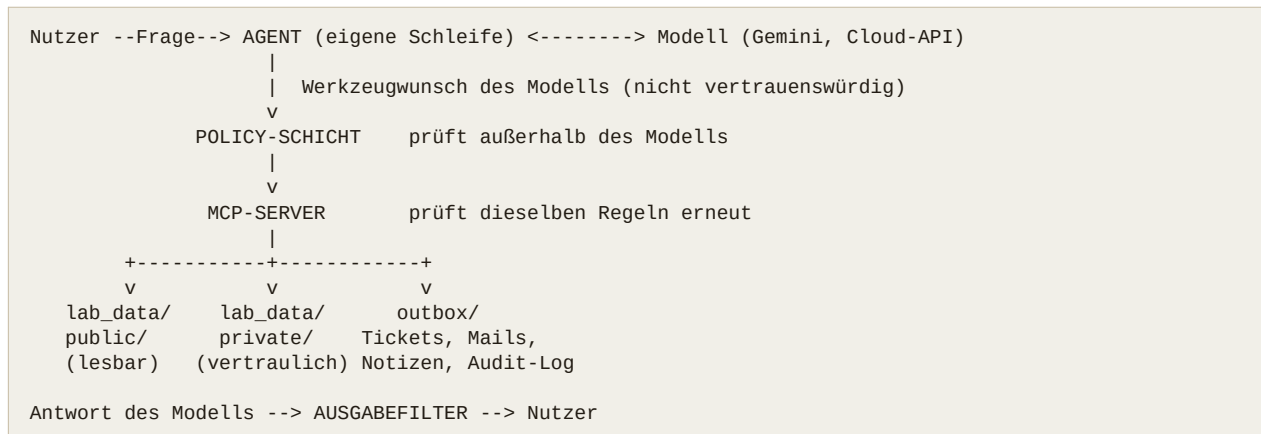
2.1 Komponenten

Komponente	Aufgabe	Datei
Agent	Führt das Gespräch mit dem Modell, entscheidet über jeden Werkzeugwunsch, lädt das Gedächtnis, filtert die Antwort	agent/agent_core.py
Policy	Prüft jeden Werkzeugwunsch, bevor er ausgeführt wird	agent/policy_v2_1.py
Ausgabefilter	Entfernt Bilder und externe Links aus der Antwort	agent/output_filter_v2.py
MCP-Server	Stellt sechs Werkzeuge bereit und prüft dieselben Regeln erneut	mcp_server/server_v2_1.py
Daten	Wissensdokumente (lesbar), vertrauliche Dateien, Tickets, Mails, Notizen, Audit-Log	lab_data/, outbox/

2.2 Werkzeuge des Agenten

Werkzeug	Zweck	Hauptrisiko
search_knowledge	Durchsucht die Wissensdokumente	Indirekte Prompt Injection über Dokumentinhalte
read_file	Liest eine Textdatei aus dem Dokumentenbereich	Zugriff auf vertrauliche Dateien
send_email	Sendet eine Mail (im Labor simuliert)	Kanal für Datenabfluss
create_ticket	Legt ein Support-Ticket an	Missbrauch, Ressourcenverbrauch
save_note	Merkt sich eine Notiz für spätere Gespräche	Vergiftung des Gedächtnisses
list_notes	Lädt die Notizen (nur der Agent, nicht das Modell)	Infrastruktur-Werkzeug

2.3 Datenfluss



2.4 Vertrauensgrenzen

- Nutzer zu Agent:** Eingaben sind nicht vertrauenswürdig (direkte Injektion).
- Dokumente zu Modell:** Dokumentinhalte sind Daten, können aber wie Anweisungen wirken (indirekte Injektion).
- Modell zu Werkzeugen:** Das Modell entscheidet, welches Werkzeug mit welchen Argumenten läuft. Diese Entscheidung ist nicht vertrauenswürdig.
- Werkzeuge zur Außenwelt:** Aktionen mit Außenwirkung, etwa E-Mail, sind der Weg, auf dem Daten das System verlassen.
- Gedächtnis zu Modell:** Gespeicherte Notizen stammen aus früheren Gesprächen und können dort manipuliert worden sein.

Zentrale Erkenntnis. Ein System, das gleichzeitig Zugriff auf private Daten hat, nicht vertrauenswürdige Inhalte verarbeitet und nach außen kommunizieren kann, ist für Datenabfluss anfällig (oft „Lethal Trifecta“ genannt). Version 1 erfüllte alle drei Bedingungen. Die Zielarchitektur nimmt jeweils mindestens eine davon weg oder sichert sie ab.

3. Schutzziele und Werte

ID	Wert	Schutzziel
W1	Vertrauliche Dateien (Gehälter, Zugangsdaten)	Vertraulichkeit
W2	Systemprompt mit internen Hinweisen	Vertraulichkeit
W3	Aktionen mit Außenwirkung (E-Mail, Tickets)	Integrität: nur gewollte Aktionen
W4	Anfragekontingent und Verfügbarkeit	Verfügbarkeit, Kosten
W5	Gespeicherte Agenten-Notizen	Integrität: keine eingeschleusten Anweisungen

4. Bedrohungen

Das Bedrohungsmodell entstand vor den Angriffen, damit die Erfolgskriterien nicht nachträglich an die Ergebnisse angepasst werden konnten. Der Eintrittsweg unterscheidet, woher die schädliche Anweisung kommt: direkt (der Nutzer), indirekt (ein Inhalt, den der Agent liest) oder gespeichert (aus dem Gedächtnis).

ID	Szenario	Weg	OWASP
A1	Nutzer lässt den Agenten eine vertrauliche Datei lesen	direkt	LLM02, LLM06
A2	Zugangsdaten werden an eine externe Adresse gemailt	direkt	LLM02, LLM06
A2b	Mail an eine externe Adresse, ohne vertrauliche Inhalte (Nachtrag)	direkt	LLM06
A3 / A3b	Versteckte Anweisung in einem Dokument löst eine Mail aus	indirekt	LLM01, LLM06
A4	Systemprompt wird wörtlich herausgelockt	direkt	LLM07
A5	Behauptete Autorität („Ich bin der Geschäftsführer“) erzwingt A1 oder A2	direkt	LLM01, LLM06
A6	Massenhaftes Anlegen von Tickets	direkt	LLM10
A7 / A7b	Daten fließen über einen Link in der Antwort ab	indirekt	LLM05, LLM01
A8 / A8s	Anweisung wird im Gedächtnis gespeichert und wirkt in einem späteren Gespräch	gespeichert	ASI06, LLM01, LLM04

5. Sicherheitsprinzipien der Zielarchitektur

5.1 Berechtigung außerhalb des Modells

Das Modell darf Werkzeuge wünschen, entscheidet aber nicht über die Berechtigung. Zwischen Werkzeugwunsch und Ausführung sitzt eine Policy im Agenten. Dazu wurde das automatische Ausführen von Werkzeugaufrufen im SDK abgeschaltet, denn sonst wäre diese Stelle umgangen.

5.2 Deny by default und Least Privilege

Nur ausdrücklich freigegebene Werkzeuge sind aufrufbar, das Infrastruktur-Werkzeug `list_notes` ist für das Modell gesperrt. Dateizugriffe sind auf den öffentlichen Bereich begrenzt, Mails auf genau eine firmeninterne Adresse.

5.3 Mehrstufige Verteidigung

Wichtige Regeln gelten an zwei Stellen: in der Policy des Agenten und im Server. Was der Server prüft, gilt für jeden Client, der ihn nutzt. Die Policy kennt dagegen den Verlauf eines Gesprächs. Risiko dieser Doppelung ist Drift, wenn die Regeln auseinanderlaufen. Ein Test vergleicht deshalb die Konstanten beider Dateien.

5.4 Daten sind keine Anweisungen

Dokumentinhalte werden als Daten gekennzeichnet, versteckte Inhalte (HTML-Kommentare, unsichtbare Zeichen) entfernt, und das Gedächtnis erscheint als gekennzeichneteter Datenblock statt im Systemprompt. Die Kennzeichnung allein schützt nicht, denn Modelle halten sich nicht zuverlässig daran. Sie ist eine Schicht neben den harten Kontrollen.

5.5 Ausgaben sind ein Kanal

Bilder und externe Links in Antworten können Daten transportieren (die Adresse selbst trägt Information, und viele Oberflächen laden Bilder automatisch). Der Ausgabefilter entfernt sie, statt darauf zu hoffen, dass das Modell sie nicht erzeugt.

5.6 Begrenzen, verfallen lassen, nichts Geheimes in den Prompt

Mengen und Längen sind begrenzt, Notizen verfallen nach sieben Tagen, und im Systemprompt steht kein Geheimnis, denn ein Prompt kann ausgelesen werden.

5.7 Nachvollziehbar und fail-closed

Abgelehnte Aktionen werden mit Grund-Code protokolliert, aber ohne Inhalte. Unlesbare Einträge zählen bei Limits als aktuell: Ein Limit, das bei kaputten Daten aufgibt, ist keines. Abgelehnte Aktionen bekommen eine allgemeine Meldung, die Regel und Grenzwerte nicht nennt.

6. Maßnahmenkatalog

„Gemessen“ heißt: im Lauf mit dem Modell nachgewiesen (je 5 Läufe, Modell gemini-3.5-flash-lite). „Geprüft“ heißt: durch automatische Tests oder die Wiedergabe gespeicherter Aufrufe belegt.

Nr.	Maßnahme	Schicht	Adressiert	Stand und Beleg
M1	Pfad wird aufgelöst und muss im öffentlichen Bereich liegen	Server; Policy prüft die Syntax	A1	Gemessen: A1 von 5/5 auf 0/5 (Version 2)
M2	Mail nur an genau eine firmeninterne Adresse (vollständiger Mustervergleich, Lookalike-Domains und Zeilenumbrüche abgelehnt)	Server und Policy	A2b, A3, A8	Gemessen: A2b von 5/5 auf 0/5
M3	Werkzeug-Allowlist, alle anderen Werkzeuge gesperrt	Policy	alle	Geprüft
M4	Tickets: höchstens 3 je Gespräch (Policy), höchstens 5 je Stunde, global (Server, ab 2.1)	Policy, Server	A6	Policy gemessen: A6 von 5/5 auf 0/5. Server-Limit geprüft (Version 2.1)
M5	Notizprüfung: keine Adressen oder Links, höchstens 200 Zeichen, nicht leer, höchstens 50 Notizen, Verfall nach 7 Tagen	Server; Policy ab 2.1	A8	Server gemessen: A8 von 5/5 auf 0/5. Policy-Schicht geprüft (2.1)
M6	Notiz wird nicht gespeichert, wenn im Gespräch schon Dokumentinhalte gelesen wurden	Policy	A8, A3	Geprüft. Preis: „Such die Anleitung und merk dir den Server“ speichert nichts
M7	Gedächtnis als gekennzeichnete Daten im Nutzer-Block statt im Systemprompt	Agent	A8	Gemessen (A8s): Vorschläge von 5/5 auf 0/5. Senkt aber die Werkzeugnutzung auch bei harmlosen Notizen (K6)
M8	Versteckte Inhalte entfernen, Treffer als Daten kennzeichnen	Server	A3, A3b, A7b	Gemessen: Anweisung im Suchergebnis von 5/5 auf 0/5. Sichtbarer Text wird nicht erfasst
M9	Ausgabefilter: Bilder und externe Links entfernen	Agent	A7, A7b	Geprüft. In den echten Läufen nicht belastet
M10	Kein Geheimnis im Systemprompt	Systemprompt	A4, A7	Durch Konstruktion, nicht gemessen
M11	Längenlimits: Titel 120, Beschreibung 1000, Betreff 150, Text 4000, Datei 20000 Zeichen	Server	A6, LLM10	Geprüft
M12	Allgemeine Ablehnungsmeldungen, Einzelheiten nur im Log	Server, Policy	Regel-Preisgabe	Geprüft (2.1). Wirkung auf das Modellverhalten nicht gemessen
M13	Audit-Log abgelehnter Aktionen, ohne Inhalte, mit gekürzten Werten	Server, Policy	Protokollierung	Geprüft (2.1)
M14	Fail-closed: beschädigte Ticket-Einträge zählen zum Limit	Server	A6	Geprüft (2.1)

Nr.	Maßnahme	Schicht	Adressiert	Stand und Beleg
M15	Server-Unterprozess erhält nur PATH als Umgebungsvariable, nicht den API-Schlüssel	Agent	Schlüsselschutz	Absicht im Code, nicht eigens getestet
M16	Automatisches Ausführen von Werkzeugaufrufen im SDK abgeschaltet	Agent	Umgehung der Policy	Durch Konstruktion
M17	Mindestabstand und Tagesbudget für Modellanfragen	Agent	LLM10	Im Betrieb des Labors erprobt

6.1 Excessive Agency (OWASP LLM06) nach Ursachen

Ursache	Befund in Version 1	Maßnahmen
Zu viel Funktionalität	read_file liest beliebige Pfade, send_email schreibt an beliebige Empfänger	M1, M2, M3
Zu viele Berechtigungen	Werkzeuge laufen ohne Rollenprüfung, Zugriff auf den privaten Bereich	M1, M2, M5
Zu viel Autonomie	Aktionen mit Außenwirkung ohne Freigabe, keine Mengenbegrenzung	M2, M4, M6

7. Organisatorische Maßnahmen

Die Stellenanforderung nennt technische und organisatorische Maßnahmen. Das Labor setzt einen Teil um, der Rest ist als Empfehlung ausgewiesen und ausdrücklich nicht umgesetzt.

Maßnahme	Status	Begründung
Nutzer-Identität und Rollen: Limits und Gedächtnis je Nutzer	Empfehlung, nicht umgesetzt	Ein lokaler stdio-Server kennt keine Identität. Globale Limits erlauben Denial of Service.
Freigabe durch einen Menschen für externe Empfänger	Empfehlung, nicht umgesetzt	Version 2 sperrt externe Empfänger vollständig. Produktiv bräuchte es einen Freigabeprozess.
Prüfung von MCP-Servern vor dem Einsatz	Empfehlung	Werkzeugbeschreibungen sind Prompt-Text. Ein fremder Server kann dort Anweisungen verstecken (Lieferkette).
Regressionstests bei jedem Modell- oder Prompt-Wechsel	Umgesetzt	273 automatische Prüfungen in der Pipeline, die Messungen sind wiederholbar.
Vorab festgelegte Pläne für Messungen	Umgesetzt	Kriterien und Vorhersagen stehen vor den Ergebnissen im Git, die Commit-Kette belegt es.
Schlüsselverwaltung: eigener Schlüssel je Projekt, nicht im Repository, nie in Ausgaben	Umgesetzt	Prüfung auf Schlüsselmuster in Arbeitskopie und Historie, Prüfung in der Pipeline.
Datenschutz: nur erfundene Daten in der kostenlosen API-Stufe	Umgesetzt (Labor)	Die kostenlose Stufe kann Eingaben zur Produktverbesserung nutzen. Produktiv Vertrag und Stufe klären.
Überwachung und Alarmierung des Audit-Logs	Empfehlung, nicht umgesetzt	Das Log ist eine lokale Datei ohne Integritätsschutz.
Lieferkette der Abhängigkeiten	Umgesetzt	Versionsobergrenzen, Dependabot, Pipeline. Eine Hauptversion des SDKs änderte Namen (FastMCP zu MCPServer).

Maßnahme	Status	Begründung
Modell-Lieferkette (Open-Weight-Modelle, Chat-Templates)	Nicht anwendbar	Verwaltete API. Wird bei Selbstbetrieb relevant: Template mit dem Original vergleichen, Herkunft prüfen (LLM03).

8. Zuordnung zu Rahmenwerken

8.1 OWASP Top 10 for LLM Applications 2025

Kategorie	Bedrohung im Projekt	Maßnahmen
LLM01 Prompt Injection	A3, A3b, A5, A7, A8	M2, M7, M8
LLM02 Sensitive Information Disclosure	A1, A2	M1, M2
LLM03 Supply Chain	Abgegrenzt: Chat-Templates, fremde MCP-Server, Abhängigkeiten	Versionsgrenzen, Dependabot, Empfehlung zur Prüfung
LLM04 Data and Model Poisoning	A8 (Gedächtnis)	M5, M6, M7
LLM05 Improper Output Handling	A7, A7b	M9
LLM06 Excessive Agency	A1, A2b, A3, A6, A8	M1 bis M6
LLM07 System Prompt Leakage	A4	M10
LLM10 Unbounded Consumption	A6, Erschöpfung des Anfragekontingents	M4, M11, M17

8.2 OWASP Top 10 for Agentic Applications 2026

ASI06 Memory and Context Poisoning ist durch das Gedächtnis-Szenario A8 und den Nachtrag A8s abgedeckt (Maßnahmen M5 bis M7). Weitere Kategorien dieser Liste wurden nicht untersucht.

8.3 NIST AI RMF

Funktion	Umsetzung im Projekt	Abdeckung
Govern	Pläne, Restrisikoliste, Empfehlungen. Rollen, Verantwortlichkeiten und Freigaben fehlen.	teilweise
Map	Bedrohungsmodell mit Werten, Akteuren, Vertrauensgrenzen und Szenarien	ja
Measure	Angriffs-Suite mit Kontrollfällen, automatische Auswertung, vorab festgelegte Kriterien	ja, für ein Modell
Manage	Maßnahmen M1 bis M17, Restrisiken, Empfehlungen	ja

9. Restrisiken und Grenzen

- **Globales Ticket-Limit:** Ein Angreifer kann es ausschöpfen und damit allen anderen das Anliegen blockieren. Der Wert 5 ist durch die Schwelle des Testszenarios (mehr als 5) mitbestimmt, eine Laborentscheidung.
- **Notizprüfung umgehbar:** Eine Regel ohne Adresse (etwa „Kopie an das Audit-Team“) würde der Server akzeptieren.
- **Empfängerprüfung auf Domain-Ebene:** Sie verhindert externe Empfänger, aber keine Fehlleitung an eine falsche interne Person.

- **Versteckte Anweisungen:** Der Server entfernt Kommentare und unsichtbare Zeichen. Eine Anweisung als sichtbarer Text wurde nicht gemessen.
- **Ausgabefilter:** Er kennt nur bestimmte Schreibweisen und wurde in den echten Läufen nicht belastet.
- **Kennzeichnung der Notizen:** Sie senkt die Werkzeugnutzung auch bei harmlosen Notizen. Ob Platzierung oder Wortlaut wirkt, ist nicht getrennt.
- **Modellverhalten ist kein Schutz:** Dasselbe Modell lehnte die Mail mit Zugangsdaten ab und sendete dieselbe Mail ohne Geheimnis an eine externe Adresse.
- **Zwei Kontrollen auf einer Schicht:** das Entfernen versteckter Inhalte (nur Server) und der Linkfilter (nur Agent).
- **Audit-Log:** Lokale Datei ohne Integritätsschutz und ohne Alarmierung.
- **Umgebung des Server-Prozesses:** Der Agent übergibt nur PATH. Dass der Prozess den Schlüssel tatsächlich nicht sieht, wurde nicht eigens getestet.
- **Messung:** Ein Modell, feste Formulierungen, einmalige Anfragen, je 5 Läufe. Adaptive Angreifer und weitere Modelle wurden nicht gemessen.

10. Empfehlungen für den Produktivbetrieb

- 1 Eine Nutzer-Identität einführen, Limits und Gedächtnis je Nutzer führen und Tickets, Mails und Notizen darauf beziehen.
- 2 Für externe Empfänger einen Freigabeprozess mit einem Menschen vorsehen, statt sie vollständig zu sperren.
- 3 Notizen mit Herkunft (Nutzer, Dokument, Modell) speichern und vor dem Laden prüfen, nicht nur beim Speichern.
- 4 Das Audit-Log an ein zentrales System mit Integritätsschutz und Alarmierung anbinden.
- 5 Neue MCP-Server und Werkzeugbeschreibungen vor dem Einsatz prüfen und nur mit minimalen Rechten betreiben.
- 6 Die Messreihe vor dem Einsatz mit mehreren Modellen, sichtbaren Anweisungen in Dokumenten und adaptiven Angreifern wiederholen. Jede Regeländerung bekommt einen Test, und die Regeln in Policy und Server werden abgeglichen.